Deterministic Optimization

Discrete Optimization: Introduction

Shabbir Ahmed
Anderson-Interface Chair and Professor
School of Industrial and Systems Engineering

Computational Complexity

Discrete Optimization: Challenges

Learning Objectives

- Discover computational complexity

Running time of Algorithms

- Given an **instance** I of a **problem** P and an **algorithm** A, we would like to understand how much **effort** the algorithm requires to solve I

- The effort is measured as the number of steps or elementary operations required by Algorithm A on the instance

- The effort required by Algorithm A of course depends on the size of the instance I

Example 1

- Problem P: Compute the scalar product of two vectors of size n
- Algorithm A:

```
def product(a,b,n):
    return sum([a[j]*b[j] for j in range(n)])
```

- Effort: Need to do n multiplications and then sum up the n products
- The effort of algorithm A scales linearly with size n of the problem instance

Example 2: Enumeration

```
# Solving an example discrete optimization problem by enumeration
import itertools
from random import randint
import time

# generate random problem instance
n = 25
r = [randint(0,9) for j in range(n)]
c = [randint(0,9) for j in range(n)]
B = sum(c)/2.0

# all possible solutions
solutions = list(itertools.product([0, 1], repeat=n))

xbest = []
vbest = -1

# check each solution
start_time = time.time()
for x in solutions:
    # check feasibility
    val = sum( [c[j]*x[j] for j in range(n)])
    if val <= B:
        obj = sum( [r[j]*x[j] for j in range(n)])
        if obj > vbest:
            vbest = obj
            xbest = x

print 'n:', n
print 'Time:', (time.time() - start_time)

```

n	Time (secs)
5	0.0012
10	0.0049
15	0.1943
20	7.4745
25	311.9747

With n=50 estimated time > 330 years!!

The effort increases exponentially (in the order of 2^n) as n increases!

Big-O Notation

- Let T(n) be the effort or running time of algorithm A on an instance of size n
- We want to understand how T(n) grows with n
- Let f(n) be a function of n
- We say that T(n) grows in “order” of f(n), i.e. T(n) = O(f(n)) if there exists constants C and N such that

$$T(n) \leq Cf(n) \text{ for all } n \geq N$$

More on Big-O

Example:
The function $T(n) = 3n^2 + 2n + 10$ is $O(n^2)$ since $T(n) \leq 15n^2 \quad \forall\ n \geq 1$

The leading terms determining the big O order

notation	name
$O(1)$	constant
$O(\log(n))$	logarithmic
$O((\log(n))^c)$	polylogarithmic
$O(n)$	linear
$O(n^2)$	quadratic
$O(n^c)$	polynomial
$O(c^n)$	exponential

Remark on Instance Size

- Note that the effort in the previous examples not only depend on n but also on the size of the numbers (since multiplying two large numbers requires more work than multiplying two small numbers)

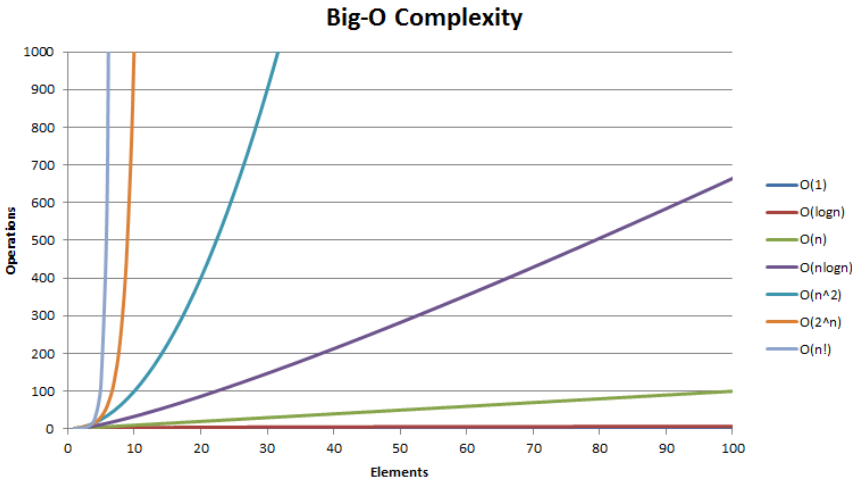
- Computers work with finite precision, so we only consider rational problem data (we can assume integers only for simplicity). Computers work with binary (or bit) representation of numbers

- Size of an instance is the total space required by the problem data

- Example: Product problem Size $\leq 2*n*\log U$ where $U = \max\{a_j,b_j\}$

Polynomial Time Algorithms

- An algorithm is polynomial time if its effort/running time T(n) as a function of the instance size n is $O(n^k)$ for some k
- Polynomial time algorithms are “efficient”, and problems for which such algorithms exists are deemed “easy”



Source:
https://i.stack.imgur.com/ia6VB.png

Problem Class P and NP

- Problem class **P**: The set of all problems for which can be solved in polynomial time (“easy” problems)

- Problem class **NP**: The set of all problems for which a given solution can be “verified” or “checked” in polynomial time

- Of course if a problem can be solved in polynomial time then we can verify a given solution in polynomial time so

$$\mathbf{P} \subseteq \mathbf{NP}$$

Example

- Partition problem:

  Given a set of n integers $a_1, \ldots, a_n$, find a subset $S \subset \{1, \ldots, n\}$ such that

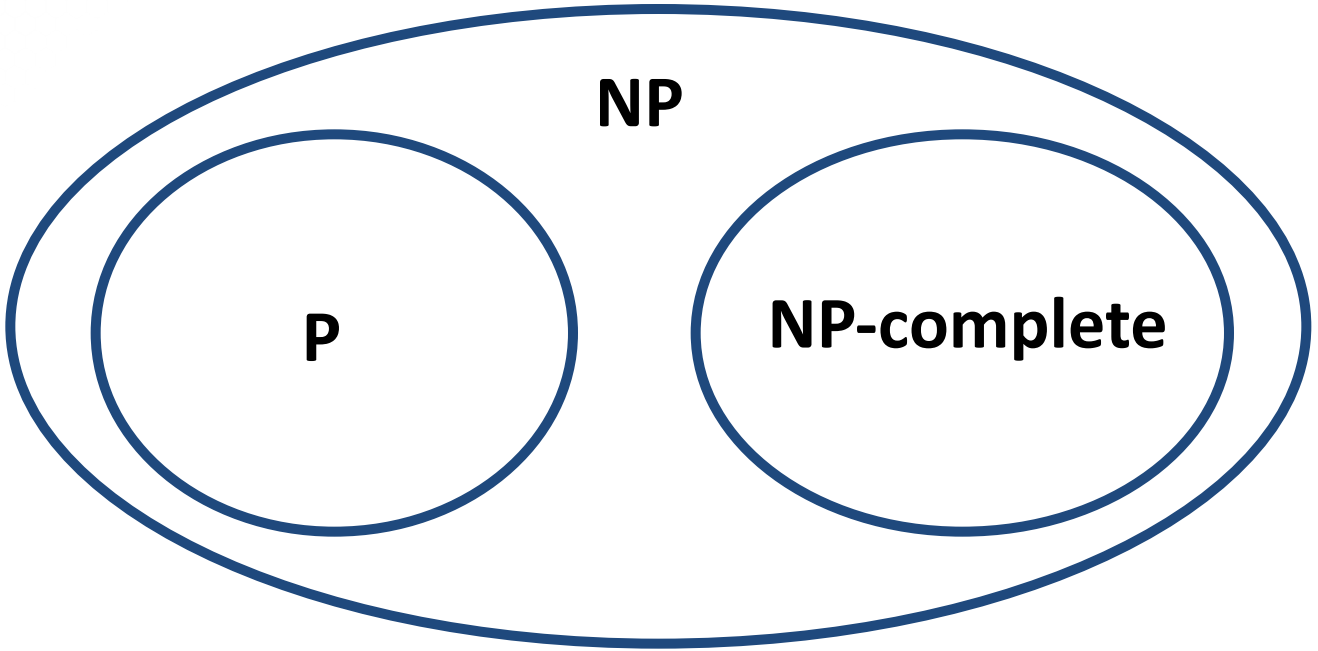
$$\sum_{j \in S} a_j = \sum_{j \notin S} a_j$$

- We do not know a polynomial time algorithm to find a solution
- However, given a set S we can easily check if it is a solution!
- So this problem is not know to be in **P** but is known to be in **NP**

Problem Class NP-Complete

- A million dollar question: Is **P** = **NP**?
- Widely believed to be false, but a formal proof is not known
- Within the set of problems in **NP** there is a set of problems such that every problem in **NP** can be “easily converted” to a problem in this set. These problems are called **NP-complete.**
- If we can show one of the problems in the **NP-complete** set is in **P** then **P** = **NP**

P, NP and NP-Complete



Problems Not in NP

- For some problems it is not easy to check whether a given solution is correct
- Example: Knapsack problem

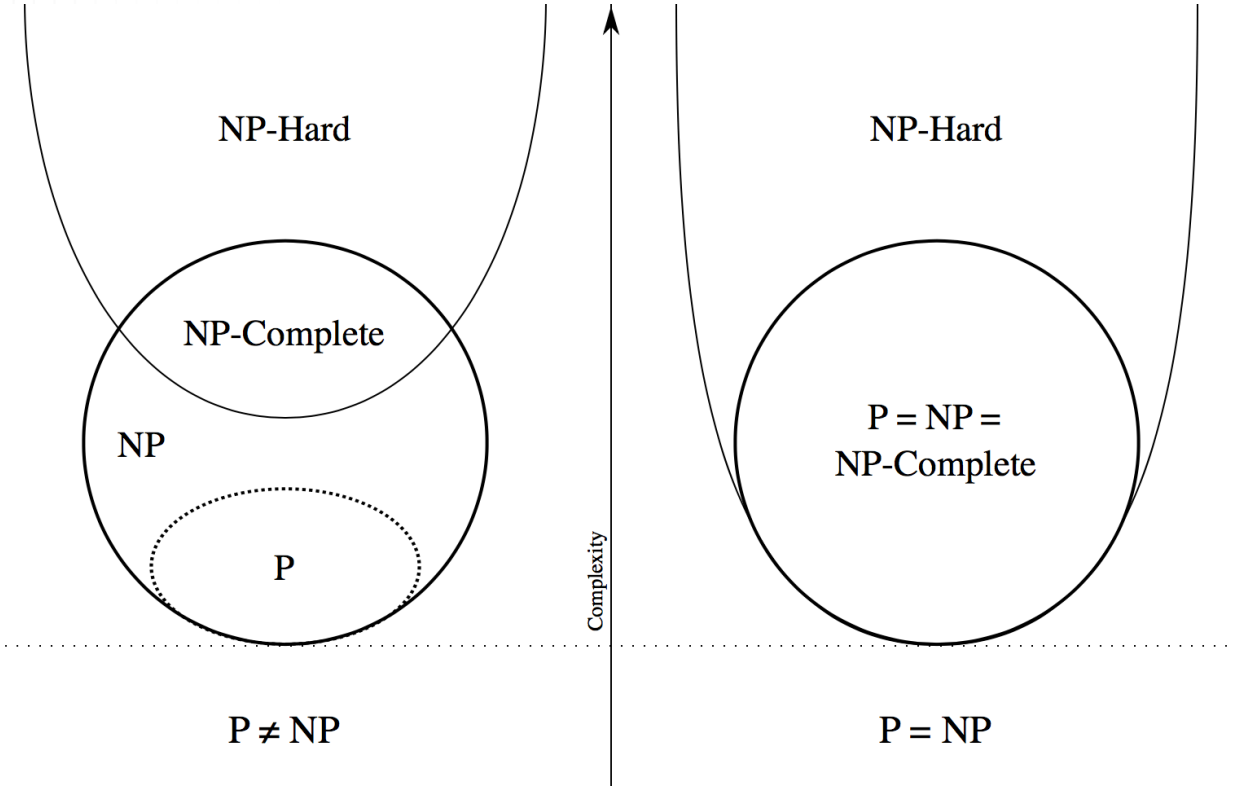
$$\begin{aligned} \max \quad & \sum_{j=1}^{n} r_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^{n} c_j x_j \leq B \\ & x_j \in \{0,1\} \; \forall \; j = 1, \ldots, n \end{aligned}$$

- Checking if a given solution is optimal is as hard as solving the problem. This problem is not in **NP**
- This is the case with most difficult optimization problems

Problem Class NP-Hard

- A problem is **NP-hard** if an algorithm for it can be converted to one for solving any problem in **NP**
- A **NP-hard** problem may not be in **NP**
- All **NP-complete** problems are **NP-hard** but not vice versa
- **NP-hard** means at least as hard as any problem in **NP**

Complexity Classes



Source:
https://en.wikipedia.org/wiki/NP-hardness#/media/File:P_np_np-complete_np-hard.svg

Optimization Problems

- Linear programming and many important classes of convex optimization problems (e.g. conic programming and semidefinite programming) are known to be in **P**

- Most discrete optimization problems are **NP-hard**

- It is widely believed that there is little hope of finding a polynomial time algorithm for discrete optimization problems

Summary

- An algorithm is polynomial time if its running time grows at a rate no more than a polynomial function of the instance size
- Problems for which polynomial time algorithms are known are easy
- Classes: P, NP, NP-complete, NP-hard
- Discrete optimization problems are NP-hard